ЗВІТ

про виконання лабораторної роботи №4

«Власні віджети та компоненти інтерфейсу»

з дисципліни

«Програмування мобільних додатків»

Студента групи КН-2327Б

Багрія-Белза Андрія

Завдання: 1. Продовжити роботу над проєктом, розпочатим у Лабораторній роботі №3.

2. На основі готового макета з Лабораторної роботи №3 переробити екран, використовуючи власні віджети.

3. Виділити елементи інтерфейсу в окремі компоненти (StatelessWidget), зокрема: заголовок/верхній інформаційний блок, інформаційну картку/текстовий блок, блок кнопок дії, блок із зображенням або іконкою.

4. Для кожного власного віджета створити окремий файл у папці widgets/ та реалізувати передачу даних через конструктор (required параметри).

5. Організувати структуру проєкту з використанням папок screens/ та widgets/.

6. У файлі екрана (lab4_screen.dart) імпортувати та використати створені власні віджети та layout-віджети (Column, Row, Container, Padding, Flexible, Spacer).

7. Забезпечити відсутність помилок верстки та перевірити роботу на емуляторі.

1 Написання коду-розв’язку завдання:

```
// === lib/screens/lab4_screen.dart ===
class _Lab4ScreenState extends State<Lab4Screen> {
  bool _isFavorite = false;

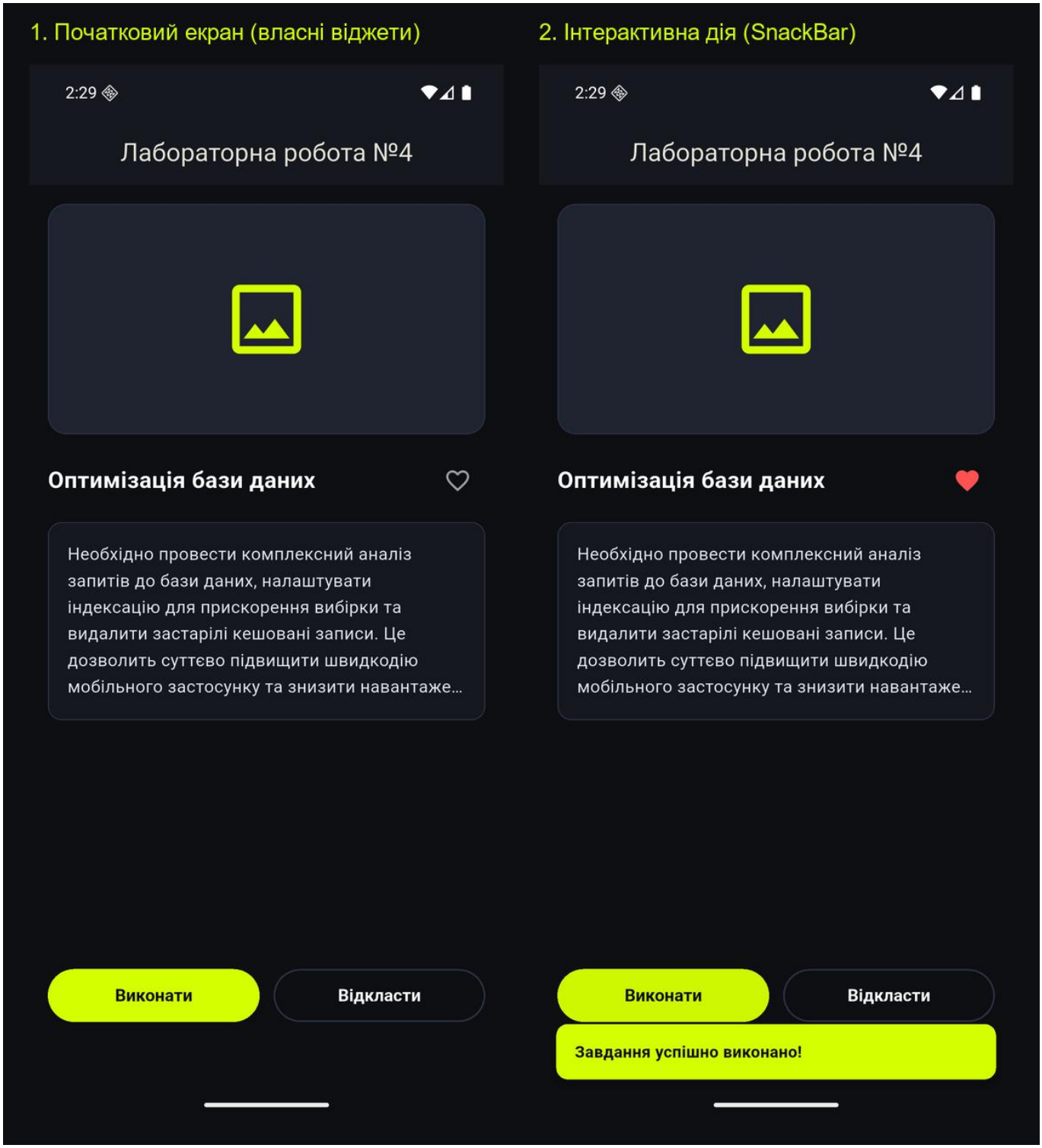
  void _showMessage(String text) {
    final messenger = ScaffoldMessenger.of(context);
    messenger.hideCurrentSnackBar();
    messenger.showSnackBar(SnackBar(content: Text(text)));
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
  // ... (пропуск неістотних рядків)
              TaskHeader(
                title: 'Оптимізація бази даних',
                isFavorite: _isFavorite,
                onFavoriteToggle: () {
                  setState(() {
                    _isFavorite = !_isFavorite;
                  });
                  _showMessage(_isFavorite ? 'Додано до обраного' : 'Видалено з обраного');
                },
              ),
  // ... (пропуск неістотних рядків)
              ActionButtonsRow(
                primaryText: 'Виконати',
                secondaryText: 'Відкласти',
                onPrimaryPressed: () => _showMessage('Завдання успішно виконано!'),
                onSecondaryPressed: () => _showMessage('Завдання відкладено'),
              ),
  // ... (пропуск неістотних рядків)
  }
}

// === lib/widgets/task_header.dart ===
class TaskHeader extends StatelessWidget {
  final String title;
```

```
  final bool isFavorite;
  final VoidCallback onFavoriteToggle;
  // ... (пропуск неістотних рядків)
  // build: Row [Expanded(Text(title)), IconButton(іконка обраного, onPressed:
onFavoriteToggle)]
  // ... (пропуск неістотних рядків)
}
```

2 Результат виконання коду:



1. Що таке власний віджет у Flutter? Власний віджет (custom widget) у Flutter — це окремий призначений для користувача клас (як правило, спадкоємець StatelessWidget або StatefulWidget), створений шляхом композиції стандартних віджетів фреймворку для інкапсуляції певної частини інтерфейсу та його логіки.

2. Для чого створюють власні компоненти інтерфейсу? Власні компоненти створюють для усунення дублювання коду (принцип DRY), спрощення структури віджетного дерева, полегшення підтримки та рефакторингу коду, підвищення читабельності та забезпечення повторного використання елементів інтерфейсу в різних частинах застосунку.
3. Чому в цій лабораторній роботі використовуються саме StatelessWidget? StatelessWidget використовується тому, що створені власні компоненти є презентаційними (відображувальними): вони не зберігають внутрішнього змінного стану, повністю залежать від переданих у конструктор даних і при їх зміні перебудовуються ззовні батьківським віджетом.
4. Як передати дані у власний віджет? Дані передаються через параметри конструктора класу власного віджета (наприклад, типи String, IconData, Color, VoidCallback) і зберігаються у незмінних (final) полях класу.
5. Для чого використовується ключове слово required у конструкторі? Ключове слово required в іменованих параметрах конструктора гарантує на етапі компіляції, що відповідний аргумент обов'язково буде переданий при створенні екземпляра віджета, запобігаючи виникненню помилок відсутності обов'язкових даних (null).
6. Чому не рекомендується розміщувати весь код у файлі main.dart? Розміщення всього коду в одному файлі призводить до надмірної складності («спагеті-код»), погіршення читабельності, ускладнення навігації, командної розробки та тестування. Файл main.dart повинен слугувати лише точкою входу для конфігурації та запуску застосунку.
7. Яке призначення папок screens та widgets у Flutter-проєкті? Папка screens/ призначена для повноцінних екранів (сторінок) застосунку, які зазвичай містять каркас Scaffold і координують взаємодію між компонентами. Папка widgets/ призначена для дрібних, автономних та повторно використовуваних UI-компонентів, з яких будуються ці екрани.
8. Які переваги має повторне використання віджетів у Flutter? Повторне використання скорочує обсяг кодової бази, забезпечує візуальну та поведінкову консистентність дизайну в усьому застосунку, прискорює розробку нових екранів і дозволяє вносити глобальні зміни в стиль чи логіку елемента лише в одному місці.
Висновок: На цій лабораторній роботі, я закріпив навички декомпозиції інтерфейсу у Flutter, виніс елементи екрана у власні StatelessWidget-компоненти (ImageBannerCard, TaskHeader, TaskInfoCard, ActionButtonsRow) із передачею даних через required-параметри конструкторів, структурував проєкт за папками screens/ та widgets/, а також перевірив коректність роботи оновленої архітектури за допомогою віджет-тестів та запуску на емуляторі Android.